

Laboration 2 – Binära sökträd och rekursiva metoder

Uppgift 1 Generics

Fråga 1

`boxTest(new Box<Integer>);` kompilerar inte eftersom konstruktorn inte anropas korrekt. I Java måste ett objekt skapas med parenteser efter konstruktorn, alltså:

`new Box<Integer>()`

Jackson använder exemplet för att visa skillnaden mellan en typ och en faktisk objektinstans. `Box<Integer>` beskriver endast datatypen, men `new Box<Integer>()` skapar själva objektet. I exemplet saknas parenteserna som behövs för att anropa konstruktorn. Därför är uttrycket inte komplett Java-syntax och programmet kan inte kompileras.

Fråga 2

Vid implementationen av ett binärt sökträd används följande generiska typ:

`<T extends Comparable<? super T>>`

Det betyder att typen `T` måste kunna jämföras med andra objekt. Detta är nödvändigt eftersom trädet behöver kunna avgöra om ett värde ska placeras till vänster eller höger i datastrukturen.

`extends Comparable` innebär att typen `T` måste kunna använda `compareTo()` för att jämföra värden. Med hjälp av jämförelsen kan trädet avgöra om ett värde är mindre än, större än eller lika med värdet i den aktuella noden.

`? super T` innebär att jämförelsen även kan göras mot `T` eller en superklass till `T`. Det gör implementationen mer flexibel och generell.

PECS-regeln står för Producer Extends Consumer Super och används inom generics för att avgöra när `extends` respektive `super` ska användas.

I detta fall används `super` eftersom `Comparable` tar emot ett objekt av en typ som ska jämföras med `T`. Det gör att fler typer kan användas på ett flexibelt sätt.

Uppgift 2 Binärt sökträd

Min implementation av ett binärt sökträd består av klassen **BinarySearchTree** samt den inre klassen **BSTNode**.

Trädet är uppbyggt av noder där varje nod innehåller ett värde samt referenser till vänster och höger barn. Den första noden i trädet kallas roten.

Strukturen bygger på principen att mindre värden placeras till vänster och större värden placeras till höger. Detta gör det möjligt att effektivt söka efter, lägga till och ta bort element.

För att förstå hur ett binärt sökträd fungerar använde jag whiteboard under lektionerna. Det gjorde det lättare att visualisera hur noderna kopplas samman och hur rekursionen arbetar steg för steg genom trädet.

Hur add fungerar

När ett nytt element läggs till jämförs värdet med den aktuella nodens värde. Beroende på resultatet fortsätter metoden rekursivt till vänster eller höger delträd tills en tom plats hittas.

Pseudokod för add

```
addRecursive(current, item)
```

```
    om current är null
        skapa en ny nod med item
        öka size med 1
        returnera den nya noden

    jämför item med current.item

    om item är mindre än current.item
        current.left = addRecursive(current.left, item)

    annars om item är större än current.item
        current.right = addRecursive(current.right, item)

    annars
        ignorera item eftersom det är en dubblett

    returnera current
```

Basfall och rekursionssteg för add

Basfallet är när **current** är **null**. Då har metoden hittat en tom position i trädet där den nya noden kan placeras.

Rekursionssteget sker när **current** inte är **null**. Metoden jämför då **item** med **current.item** och fortsätter rekursivt nedåt i vänster eller höger delträd beroende på resultatet från **compareTo()**.

Om värdena är lika behandlas det som en dubblett och inget nytt värde läggs till.

Hur remove fungerar

Vid borttagning av ett element söker metoden först efter rätt nod. Därefter hanteras tre olika fall beroende på hur många barn noden har.

Pseudokod för remove

```
removeRecursive(current, item)
```

```
    om current är null  
        returnera null
```

```
    jämför item med current.item
```

```
    om item är mindre än current.item  
        current.left = removeRecursive(current.left, item)  
        returnera current
```

```
    om item är större än current.item  
        current.right = removeRecursive(current.right, item)  
        returnera current
```

```
    annars  
        om current saknar vänster och höger barn  
            returnera null
```

```
        om current saknar vänster barn  
            returnera current.right
```

```
        om current saknar höger barn  
            returnera current.left
```

```
        minsta = hittaMinsta(current.right)  
        current.item = minsta.item  
        current.right = removeRecursive(current.right, minsta.item)
```

```
        returnera current
```

Basfall och rekursionssteg för remove

Basfallet är när **current** är **null**. Då betyder det att elementet inte finns i trädet och det finns därför inget mer att ta bort.

Rekursionssteget sker när det sökta värdet inte har hittats. Metoden fortsätter då rekursivt nedåt i vänster eller höger delträd beroende på resultatet från **compareTo()**.

När rätt nod hittas hanteras tre olika fall. Om noden saknar barn tas den bort. Om noden har ett barn ersätts noden med sitt barn. Om noden har två barn ersätts värdet med det minsta värdet i höger delträd och den noden tas sedan bort.

Dubbletter

add, **search** och **remove** använder samma jämförelselogik med **compareTo()** för att navigera genom trädet. Om **compareTo()** visar att värdet är mindre går metoden åt vänster och om värdet är större går den åt höger. Om **compareTo()** returnerar **0** betyder det att värdena betraktas som lika.

Vid **add** behandlas ett lika värde som en dubblett. Då läggs värdet inte till igen. Trädet innehåller därför endast ett exemplar av varje värde.

search använder samma jämförelse och kommer därför att hitta värdet på samma plats där **add** placerade det. **remove** använder också samma jämförelse för att hitta rätt nod innan den tas bort.

Metoderna är därför kompatibla eftersom de hanterar mindre, större och lika värden på samma sätt. En krock, alltså när två värden jämförs som lika, leder till samma resultat i alla metoder: **add** lägger inte till en ny nod, medan **search** hittar det befintliga värdet och **remove** kan ta bort det.

Uppgift 3 Tidskomplexitet

Hypotes

Jag tror att sökning går snabbare när talen är slumpade eftersom trädet då blir mer balanserat.

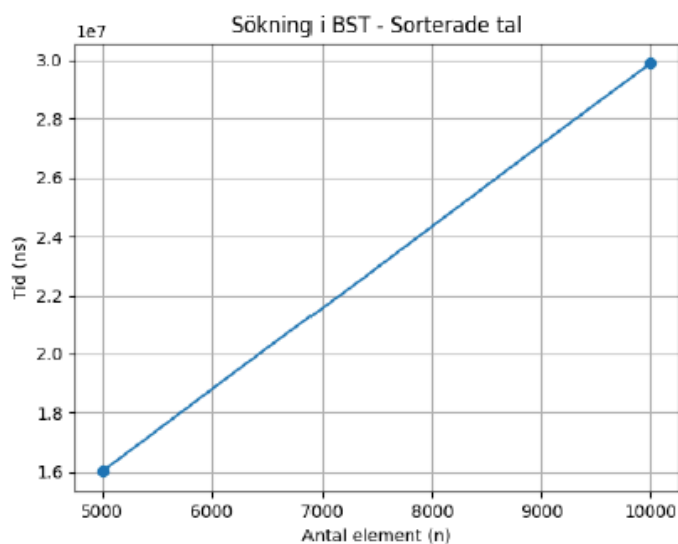
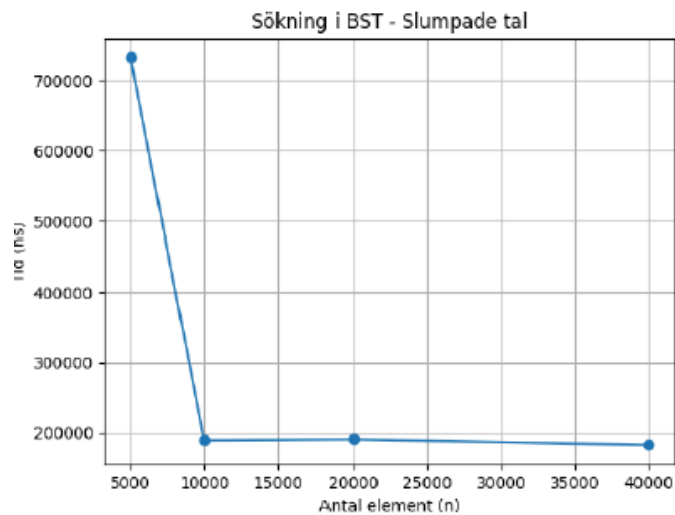
Jag tror att sorterade tal kommer ge sämre prestanda eftersom trädet blir obalanserat.

Resultat

1	n	Slumpade	Sorterade
2	5000	353800	18846200
3	10000	157600	38898300
4	20000	152200	failed
5	40000	143800	failed

Kvot

1	n	Kvot_Slumpade	Kvot_Sorterade
2	10000/5000	0.45	2.06
3	20000/10000	0,97	failed
4	40000/20000	0,94	failed



Analys

För slumpade tal minskar exekveringstiden något när antalet element ökar från 10 000 till 40 000. Tiden ligger därefter på ungefär samma nivå, vilket tyder på att trädet är relativt balanserat och att sökningen kan utföras effektivt.

För sorterade tal ökar exekveringstiden däremot kraftigt när antalet element ökar. Vid 20 000 och 40 000 element kunde programmet inte slutföra körningen. Detta beror på att trädet blir mycket obalanserat och börjar likna en länkad lista. Sökningen behöver då gå igenom betydligt fler noder.

Resultaten stämmer därför väl överens med den förväntade tidskomplexiteten för ett binärt sökträd. För ett relativt balanserat träd är sökning ungefär $O(\log n)$, medan ett obalanserat träd i värsta fall kan få tidskomplexiteten $O(n)$.

Slutsats

Resultaten stödjer hypotesen att ett binärt sökträd fungerar bättre när elementen läggs in i slumpmässig ordning. De slumpade talen ger ett relativt balanserat träd, vilket leder till effektivare sökning.

När elementen i stället läggs in i sorterad ordning blir trädet obalanserat och får en struktur som liknar en länkad lista. Detta leder till betydligt längre exekveringstider och att programmet inte klarar de största testfallen.

Experimentet visar därmed att **trädets struktur har stor betydelse för sökningens tidskomplexitet och prestanda**. Resultaten överensstämmer med teorin om att sökning kan vara $O(\log n)$ i ett relativt balanserat BST, men $O(n)$ i ett kraftigt obalanserat BST.